

Eric Faltermeier
3053092
Monday 8:00AM to 9:50AM Sumaiya Shomaji
9/13/2026

File ["/tmp/ipykernel_3463/1095460817.py"](#), line 3

Monday 8:00AM to 9:50AM Sumaiya Shomaji

^

SyntaxError: invalid decimal literal

Next steps: [Explain error](#)

```
import random
import time
import numpy as np
def main():
    while True:

        # gets which task to be caculated by the user
        whichTask = int(input("""Enter the specified number for the wante
                                0 for task1A
                                1 for task1B
                                2 for task2
                                3 to exit
                                """))

        # 3 indicates the end of the program
        if whichTask == 3:
            break

        # gets the array from the user
        inputArr = list(map(int, input("Enter the unsorted array separate

        # calls the according function based on user input
        match whichTask:
            case 0:
                taskOneA(inputArr, False)

            case 1:
                print(taskOneB(inputArr))

            case 2:
                print(taskTwo(inputArr))

        # implements insertion sort with a second "sorted array"
        def taskOneA(inputArr, isExperiment):
```

```
# holds the sorted array
sortedArr = []

# gets each number from the input array
for number in range(len(inputArr)):
    key = inputArr[number]

    # finds where the number belongs
    for i in range(len(sortedArr)):

        if key <= sortedArr[i]:
            sortedArr.insert(i, key)
            break

    # if the number is bigger than everything in sortedArr
    else:
        sortedArr.append(key)

    # shows sorted array after each iteration if its not the experiment
    if (isExperiment == False):
        print(sortedArr)

return sortedArr

# implements insertion sort by shifting values in the input array
def taskOneB(inputArr):

    # iterates through the sorted index
    for sortedIndex in range(1, len(inputArr)):
        # gets the key and sets the sorted array index (j)
        key = inputArr[sortedIndex]
        j = sortedIndex - 1

        # iterates through the sorted portion of the array until the key
        while j >= 0 and inputArr[j] > key:
            inputArr[j + 1] = inputArr[j]
            j -= 1

        inputArr[j + 1] = key

    # returns the newly sorted input array
    return inputArr

# implements merge sort on an array
def taskTwo(inputArr):

    # splits the array into two equal halves
    middle = len(inputArr) // 2
```

```

# this is the base case of the algorithm
if len(inputArr) <= 1:
    return inputArr

# splits the array in two
leftHalf = inputArr[:middle]
rightHalf = inputArr[middle:]

# gets the lengths of the arrays
leftLen = len(leftHalf)
rightLen = len(rightHalf)

# recursively splits the halves until they're ready to be sorted
if leftLen != 1:
    leftHalf = taskTwo(leftHalf)

if rightLen != 1:
    rightHalf = taskTwo(rightHalf)

# creates new indices for the arrays
i = 0
j = 0
sortedArray = []

# compares the values of the arrays and sorts them correctly
while i < leftLen and j < rightLen:
    if leftHalf[i] <= rightHalf[j]:
        sortedArray.append(leftHalf[i])
        i += 1
    else:
        sortedArray.append(rightHalf[j])
        j += 1

if i != leftLen:
    while i != leftLen:
        sortedArray.append(leftHalf[i])
        i += 1
elif j != rightLen:
    while j != rightLen:
        sortedArray.append(rightHalf[j])
        j += 1

return sortedArray

main()

# -----
# Experiment
# -----
# 1. Create a large random array
N = 5000 # you can change size (e.g., 10_000)
arr = [random.randint(0, 10**6) for _ in range(N)]

```

```
# 2. Time Insertion Sort
arr_copy = arr[:]
start = time.perf_counter()
output = taskOneA(arr_copy, True)
t_insertion = time.perf_counter() - start
print(f"Insertion Sort time: {t_insertion:.6f} sec")
# 3. Time Merge Sort
arr_copy = arr[:]
start = time.perf_counter()
output_m = taskTwo(arr_copy)
t_merge = time.perf_counter() - start
print(f"Merge Sort time: {t_merge:.6f} sec")
# 4. Verify correctness against numpy sort (and measure numpy time)
arr_copy = arr[:]
start = time.perf_counter()
np_sorted = np.sort(arr_copy) # numpy's highly optimized sort (timsort/quicksort)
t_numpy = time.perf_counter() - start
print(f"NumPy Sort time: {t_numpy:.6f} sec")
```

```
3 to exit
2
Enter the unsorted array separated by spaces: 7 7 7 7
[7, 7, 7, 7]
Enter the specified number for the wanted task:

0 for task1A
1 for task1B
2 for task2
3 to exit
2
Enter the unsorted array separated by spaces: 5 4 3 2 1
[1, 2, 3, 4, 5]
Enter the specified number for the wanted task:

0 for task1A
1 for task1B
2 for task2
3 to exit
3
Insertion Sort time: 0.283104 sec
Merge Sort time: 0.010836 sec
NumPy Sort time: 0.000503 sec
```